

Terraform

classmate

Date _____

Page _____

42

→ 'Terraform' means changing a planet's atmosphere to human-living-form/habitable

Ex - We want to change Mars atmosphere such that humans can live there.

→ For Devops: 'Terraform' means making the environment, cloud services more habitable for our project

& maintaining

- Manage Infrastructure on Cloud by Infrastructure as Cloud.

Steps to execute: [IFVUPAD]

terraform init [Install every dependency required]



terraform fmt [Properly formats all of your files]



terraform validate [Checks & give errors if any]



terraform plan [Show how it will be executed & what will be the output without actually changing the resource]



terraform apply [Actual execution]



terraform destroy [To destroy everything you created]

In Terraform Registry, There are built module that make writing Terraform code ~~is~~ ~~easy~~ ~~peasy~~, easy peasy!

Terraform Files:-

- 1.] **main.tf** → File where actual infrastructure is defined.
- 2.] **provider.tf** → Cloud Provider detail like AWS, Azure etc
- 3.] **variable.tf** → Defines all the variable
- 4.] **terraform.tfvars** → Actual value of variable given in this file.
- 5.] **output.tf** → To print output on console
like This is public IP etc
- 6.] **terraform.tfstate** → All the data of the infrastructure is stored on this file
- 7.] **terraform.tfstate.backup** → Backup of tfstate file
- 8.] **backend.tf** → Stores the state file remotely for CI/CD.
- 9.] **version.tf** → Control required version needed to run & create cloud resources.
- 10.] **terraform.lock.hcl** → Autogenerated
→ Lock provider version ensuring consistent setup.

Terraform Syntax

→ Terraform uses HCL [Hashicorp Configuration Language]

- similar to YAML & JSON

→ Anatomy:

<block-type> "provider-type" "name"

```
{ key1 = "value1"
  key2 = "value2"
}
```

Eg:- maint.tf [define resource]

```
resource "aws_instance" "kops"
```

```
{ ami = var.ami
  instance_type = "t2.micro"
}
```

Eg:- variable.tf

```
variable "ami"
```

```
{ type = string
  description = "AMI ID"
}
```

```
variable "instance_type"
```

```
{ type = string
  default = "t2.micro"
}
```

Module $\rightarrow$ A folder containing all the .tf files which increases reusability.
Best Practice: keep one module per resource [ec2, rds, vpc, etc.]
 $\rightarrow$ Make it reusable & flexible

Variables in Terraform:

$\rightarrow$ Define variables in variables.tf file.

In that file:-

variable "instance-type" {

description = "EC2 instance type:"

type = string [OPTIONAL]

default = "t3.micro"

$\rightarrow$ Now in main.tf:

Accessed using 'var.'

resource "aws_instance" "example" {

instance_type = var.instance_type

}

Note: When you do terraform apply

- It accesses all the files in the given folder to complete the execution, no need to link files in HTML, CSS & JS

- Terraform treats all files as one big configuration

- It scans every file in the folder

$\rightarrow$ If .tf files in different folders, we can use it by giving

→ Terraform also provides different workspace for different environment.

classmate

Date

Page 44

directory path.

Provisioners:-

→ They are the Last Resort

→ Some services cannot be managed by Terraform, for which 'provisioners' can be used.

Use case:- Initial configuration of a VM when no other automation is possible.
- Triggering deployment scripts after instance creation.

→ Better to use tools like Packer, Ansible than provisioners as they are the last resort. because Terraform cannot manage provisioners.

Output Information:-

→ All information stored in `terraform.tfstate` file.

→ Use output block to print these attributes.

Backend Push

→ One issue: All files remain in local machine.

→ Terraform solved it by:- backend.tf

which can push all the output files to S3 bucket, HCP terraform

↳ Terraform storage service.